

# **Autómatas, Teoría de Lenguajes y Compiladores**

## **Proyecto Especial**

Instituto Tecnológico de Buenos Aires



2025

### Integrantes:

- Nahuel Prado (64276)
- Lorenzo Chiossone (64359)
- Mateo Buela (64680)

### Docentes:

- Mario Agustin Golmar
- Ana Arias Roig

# **Índice**

|                               |   |
|-------------------------------|---|
| 1. Introducción               | 3 |
| 2. Modelo Computacional       | 3 |
| 2.1. Dominio                  | 3 |
| 2.2. Lenguaje                 | 3 |
| 3. Implementación             | 4 |
| 3.1. Frontend                 | 4 |
| 3.2. Backend                  | 5 |
| 3.4. Dificultades Encontradas | 5 |
| 4. Futuras Extensiones        | 6 |
| 5. Conclusiones               | 6 |
| 7. Bibliografía               | 6 |

# 1. Introducción

El siguiente documento explica el diseño y la implementación del compilador AZM86. Podrán encontrar información tanto del modelo conceptual usado como las distintas etapas de implementación siendo estas Frontend y Backend.

---

## 2. Modelo Computacional

### 2.1. Dominio

Desarrollar un compilador que traduzca programas escritos en el lenguaje ensamblador del procesador Zilog Z80 al lenguaje ensamblador x86-64 de Intel/AMD.

El compilador debe modelar con precisión las capacidades y limitaciones de la arquitectura del procesador Z80, asegurando que cualquier instrucción inválida para dicho procesador sea detectada y genere un error de compilación.

Por otro lado, se deben despreciar ciertas sentencias obsoletas como **ORG** o **ASEG**, que carecen de equivalencia directa o relevancia en el entorno x86-64 moderno.

Se debe mantener la lógica de las flags, así como la coherencia entre los códigos pre y post compilación, aunque esto no quiere decir que estas se mantengan con el mismo nombre, ya que se adaptarán al nuevo lenguaje.

El compilador también debe mantener soporte para la creación y uso de **macros**, permitiendo que el programador defina secuencias reutilizables de instrucciones que sean correctamente traducidas al conjunto objetivo.

Gracias a esta solución, se podrá migrar y reutilizar código del Z80 hacia plataformas modernas de 64 bits.

### 2.2. Lenguaje

El lenguaje implementado en este compilador es un subconjunto estructurado del ensamblador Z80, extendido con reglas formales que permiten su traducción al entorno x86-64. El mismo define instrucciones válidas, registros, operandos, modos

de direccionamiento y reglas sintácticas que determinan la forma correcta de los programas.

Conjunto de instrucciones soportadas:

- |       |        |        |
|-------|--------|--------|
| • ADD | • OR   | • PUSH |
| • SUB | • XOR  | • POP  |
| • INC | • CPL  | • CALL |
| • DEC | • CP   | • RET  |
| • NEG | • JP   | • NOP  |
| • LD  | • JR   |        |
| • AND | • DJNZ |        |

Registros soportados:

- |     |      |      |
|-----|------|------|
| • A | • H  | • AF |
| • B | • L  | • SP |
| • C | • BC | • IY |
| • D | • DC | • IX |
| • E | • HL |      |

---

## 3. Implementación

### 3.1. Frontend

El frontend del compilador se diseñó para transformar el código ensamblador Z80 en una representación interna que pueda ser procesada por el backend. Para ello se utilizó **Flex** para el análisis léxico y **Bison** para el análisis sintáctico.

Se definieron los **tokens básicos** del lenguaje, incluyendo registros de 8 y 16 bits (A, B, HL, IX, IY), instrucciones (LD, ADD, JP, NOP, etc.), banderas (NZ, C, Z, etc.), declaraciones de datos (DB, DW, DS), símbolos auxiliares como comas, paréntesis y etiquetas. Cada token tiene asociada una **acción léxica** que construye nodos del AST o estructuras intermedias, permitiendo un manejo consistente de los elementos del lenguaje. Además, se implementaron patrones reutilizables para enteros, identificadores y comentarios, facilitando la extensión de la gramática.

Por otro lado se definieron los **tokens no terminales**, representando estructuras de mayor nivel: el programa completo (program), bloques de código y datos (codeBlock y dataBlock), líneas individuales de código o datos (codeLine y dataLine), instrucciones (instruction) y operandos (operand), incluyendo modos de direccionamiento como registros, memoria y valores inmediatos. También se definieron producciones para macros, que permiten definir bloques reutilizables con parámetros opcionales.

Las **acciones semánticas** fueron modularizadas mediante funciones como Z80MakeCodeLineInsn o Z80ExprListAppend, que encapsulan la creación de nodos del AST. Cada producción del parser construye fragmentos del árbol sintáctico que luego se combinan para formar la estructura completa del programa.

## 3.2. Backend

El backend del compilador toma el AST generado por el frontend y produce como salida código ensamblador x86-64, actuando como la etapa de síntesis que traduce cada construcción del lenguaje Z80 a su equivalente en la arquitectura destino. Para ello se recorre el árbol y se aplica, para cada instrucción soportada, un patrón de traducción que preserva la semántica original en el marco del subconjunto definido del lenguaje.

Para simplificar la generación, se definió una capa intermedia que abstrae operaciones aritmético-lógicas, saltos y accesos a memoria antes de emitir el código textual x86-64. Esta capa reutiliza la información recolectada durante el análisis (etiquetas, constantes, macros) para resolver direcciones de salto, expandir macros en el punto correcto.

El mapeo entre registros Z80 y x86-64 se resolvió fijando asociaciones entre registros de 8 y 16 bits y registros de propósito general de 64 bits, utilizando extensiones. En los casos donde ciertas flags o instrucciones del Z80 no poseen equivalente directo, se emplean secuencias auxiliares para emular su comportamiento cuando es posible.

### **3.4. Dificultades Encontradas**

Aquí se documentan los inconvenientes técnicos y conceptuales enfrentados durante el proyecto, tales como:

- Dificultad en la traducción de los registros ya que son de distinto tamaño en cada lenguaje. Esto conlleva que no se pueda registrar fácilmente ciertas flags, decidimos no implementarlas.
  - Las operaciones no son literales uno a uno.
  - Dificultad de mapear las flags de z80 con x86, se tomó la decisión de que el registro F de z80 es un registro más y para ciertas operaciones soportadas se puede utilizar el sistema de flags de x86-64, de esta forma teniendo ciertos jumps condicionales, como el djnz.
- 

## **4. Futuras Extensiones**

Más allá de que el compilador final es completamente funcional hay ciertas extensiones que se le podrían agregar a futuro. Por ejemplo la implementación de carry, half carry y adaptar instrucciones como el jump condicional a este tipo de situaciones. También falta contemplar casos como el overflow. Otra cuestión es adaptar correctamente el registro F para que refleje correctamente las flags.

---

## **5. Conclusiones**

El desarrollo de AZM86 nos permitió profundizar en la arquitectura del Z80 y en cómo adaptarla a un entorno moderno como x86-64. A lo largo del proyecto logramos diseñar un lenguaje claro, construir un frontend capaz de generar tokens y

un AST correcto, y desarrollar un backend que traduce instrucciones y mantiene la lógica esencial del programa original. Los problemas más grandes estuvieron en las diferencias entre registros, tamaños de datos y flags, pero se pudieron resolver con un diseño cuidadoso.

Aunque el compilador funciona correctamente, quedaron ideas para futuras mejoras. Aun así, el trabajo realizado nos dejó una base sólida, un buen entendimiento del dominio y una herramienta útil que podría crecer en próximas versiones.

---

## 7. Bibliografía

### **Cartilla Assembler Z80**

(s.f.). *Cartilla Assembler Z80* (PDF).

### **Cartilla Assembler Intel**

(s.f.). *Cartilla Assembler Intel* (PDF).

### **Material de la cátedra:**

1.  (2025-03-13, v3.0.6) Proyecto Especial
2.  (2025-03-13, v0.1.0) Modelo Computacional
3.  (2024-09-25, v0.2.0) Análisis Léxico
4.  (2024-05-08, v0.1.0) Análisis Sintáctico
5.  (2024-05-08, v0.1.0) Análisis Sintáctico
6.  (2024-09-25, v0.2.0) Análisis Semántico
7.  (2024-05-16, v0.1.0) Generación de Código